GameAgent: Voyager Minecraft 运行与代码修复报告

基于 MineDojo/Voyager 的工程化分支

2026-04-29

摘要

本报告总结了当前 GameAgent 仓库中 Voyager 分支的真实运行结果、关键代码改动、稳定性修复，以及本次实验结束时能够从日志与 checkpoint 中确认的终止原因。该分支的主要目标是在真实 Minecraft 1.19 LAN 世界中持续运行 Voyager，并通过 OpenAI-compatible 接口接入 `opencode.ai/zen/go/v1`。截至当前快照，系统已经完成 27 个任务，学得 28 个技能，进度推进到铁镐、铁靴与基础洞穴资源阶段。

1 项目背景与目标

本仓库基于上游 MineDojo/Voyager，但目标与原始演示仓库不同，重点在于：

- 让 Voyager 在真实 Minecraft LAN 世界中连续运行，而不是仅作为论文代码示例。
- 使用 OpenAI-compatible API，而非固定依赖 OpenAI 官方端点。
- 保留断点续跑、长时间日志监控、以及运行稳定性修复。
- 输出可交付的工程文档与运行结果报告。

本次使用的核心入口脚本为 `run_voyager.py`，通过 `.env.local` 读取环境变量，目标模型为 `kimi-k2.6`，API Base 为 `https://opencode.ai/zen/go/v1`。

2 关键代码修改

表 1 总结了本分支中最重要的工程性改动。

表 1: 关键修改文件与作用

文件	作用
`run_voyager.py`	统一加载 `.env.local`，清理代理环境变量，配置 OpenAI-compatible 端点，并将环境步进超时提高到 300 秒。
`run.py`	去除硬编码密钥，改为从环境变量读取配置，作为短轮次 smoke test 入口。

文件	作用
`voyager/utils/llm_utils.py`	为 LLM 调用增加应用层重试和外层硬超时，避免模型调用无异常但长期卡死。
`voyager/utils/fake_embeddings.py`	当使用自定义 OpenAI-compatible 接口时启用 `FakeEmbeddings`，避免 embedding 兼容性问题。
`voyager/agents/action.py`	Action agent 改用统一的重试/超时逻辑。
`voyager/agents/curriculum.py`	Curriculum、QA、decomposition 全部改用统一 LLM 调用包装。
`voyager/agents/critic.py`	Critic 调用统一到应用层重试逻辑。
`voyager/agents/skill.py`	Skill 描述生成统一到应用层重试逻辑。
`voyager/env/bridge.py`	增强 `/step` 与 `/start` 的恢复能力，mineflayer 重启后会进行重试，不再因为短暂竞态直接终止整个学习流程。
`voyager/env/mineflayer/index.js`	修复 `/start` 生命周期空指针问题，修复 stuck recovery 中对空 block 列表错误取值，增强 `/step` 阶段的局部 bot 捕获。
`voyager/control_primitives/mineBlock.js`	增加掉落等待和二次收集逻辑，减轻“方块已挖但掉落尚未入包”的情况。

3 运行结果

截至当前 checkpoint，系统已经完成下列量化结果：

- 已完成任务：27 个
- 已失败任务：3 个
- 已学得技能：28 个

3.1 已完成任务

1. Mine 1 wood log
2. Mine 3 spruce_log
3. Craft 4 spruce_planks
4. Craft 1 crafting_table
5. Craft 8 spruce_planks
6. Craft 1 wooden_pickaxe
7. Mine 3 cobblestone
8. Mine 1 coal_ore

9. Craft 4 sticks
10. Craft 1 stone_pickaxe
11. Mine 1 copper_ore
12. Mine 8 cobblestone
13. Craft 1 furnace
14. Smelt 4 raw_copper
15. Mine 1 lapis_ore
16. Mine 4 coal_ore
17. Mine 1 iron_ore
18. Craft 1 stone_sword
19. Equip stone_sword
20. Mine 2 iron_ore
21. Smelt 3 raw_iron
22. Craft 1 iron pickaxe
23. Mine 3 iron_ore
24. Mine 4 iron_ore
25. Smelt 4 raw_iron
26. Craft 1 iron_boots
27. Equip iron_boots

3.2 失败任务

- Craft 12 spruce_planks
- Mine 1 spruce_leaves
- Kill 1 skeleton

3.3 技能库结果

当前已学得技能数为 28，覆盖了木制工具、石制工具、熔炉、煤/铜/铁/青金石挖掘、以及铁靴装备等流程。代表性技能包括：

- `craftWoodenPickaxe`
- `craftStonePickaxe`
- `craftFurnace`
- `craftIronPickaxe`
- `mineThreeCobblestone`
- `mineCoalOre`
- `mineCopperOre`
- `mineLapisOre`
- `smeltFourRawCopper`
- `smeltFourRawIron`

3.4 最后状态快照

从 `voyager_live.log` 尾部可见的最新状态如下：

- Biome: `taiga`
- Position: `x=672.5, y=69.0, z=-399.5`
- 已装备物品：`iron_boots`
- 关键库存：`iron_pickaxe: 1`、`stone_sword: 1`、`lapis_lazuli: 6`、`raw_iron: 5`、`copper_ingot: 4`、`coal: 2`、`cobblestone: 66`

这说明本次运行已经稳定推进到铁工具与基础铁护甲阶段。

4 终止原因分析

根据现有日志，能够明确确认的最后一个内部失败点发生在任务 `Kill 1 skeleton` 期间。日志中出现了如下链条：

- Python 侧通过 `voyager/env/bridge.py` 调用本地 mineflayer HTTP 服务 `/step`。
- 该请求在本地回环地址 `127.0.0.1:3000` 上触发了 `ReadTimeout`。

- 当时 `env_request_timeout` 已提升到 300 秒，说明这是一次长时间未返回的环境执行步骤，而不是短超时误判。
- 随后 Python 抛出 `RuntimeError: Failed to step Minecraft server`。
- Voyager 按设计将 `Kill 1 skeleton` 标记为失败，并继续进入下一轮 curriculum 问答流程。

因此，可以把“最后一个明确记录到的内部故障”定义为：

> 在 `Kill 1 skeleton` 任务中，本地 mineflayer `/step` 调用在 300 秒后超时，导致该任务失败。

但是，日志尾部并没有留下另一个明确的 Python fatal traceback、`Interrupted by user`、或标准的“正常退出”标记。因此，“为什么整个进程最终停了”这一点无法从现有日志 100% 精确复原。保守结论是：

- 已知内部失败是 skeleton 任务的 `/step` 超时。
- 在这之后，程序仍继续输出了一段 curriculum QA 与状态快照。
- 整体停止更像是外部中断、会话结束、或日志未覆盖到的进程终止，而不是一个已经被完整记录的新代码异常。

5 本次修复的效果

从实际结果看，这些代码改动显著改善了运行稳定性：

- 解决了早期 crafting/mineflayer 序列化问题，使木板、工作台、木镐等关键前置任务可以完成。
- 解决了自定义模型接口上“请求挂起但不报错”的问题，使 Action/Curriculum/Critic/Skill 四类 agent 都能继续推进。
- 解决了 mineflayer 生命周期中的空指针问题，降低 bot 重启过程中的二次崩溃概率。
- 通过提升环境超时与 `/start` 重试，减少了长时间地下/水下任务导致整轮学习直接退出的概率。

如果没有这些修复，本次运行基本不可能推进到铁镐与铁靴阶段。

6 剩余问题与建议

当前仍可见的问题主要有三类：

- **战斗类任务稳定性不足**：`Kill 1 skeleton` 依然会在复杂环境下触发长时间 `/step` 卡住。

- **Critic fenced JSON**：critic 仍然经常先返回带代码围栏的 JSON，导致额外一次解析失败与重试。
- **mineflayer 运行时噪声**：日志中仍存在大量实体弃用 warning 和偶发连接断开提示，虽然不再总是致命，但会增加排障成本。

下一步建议：

1. 优先增强战斗任务的动作超时与回退策略，特别是 skeleton 近战接近逻辑。
2. 在 JSON 解析器中直接处理 fenced JSON，减少 critic 的无效重试。
3. 对 `mineBlock` 和战斗任务分别增加更明确的成功/失败判据，避免“动作已结束但结果未达成”的模糊状态。

7 结论

本次分支已经从“不稳定的论文代码运行环境”演化为“可在真实 Minecraft LAN 世界中持续推进的工程化版本”。虽然最终运行没有完整达到所有课程目标，但它已经真实完成 27 个任务、学习 28 个技能，并稳定推进到铁工具和基础铁护甲阶段。`Kill 1 skeleton` 仍是当前最明显的稳定性短板，但从现有代码与结果看，系统已经具备继续迭代和继续跑更长任务链的基础。